

SmartSpace - Source Code

Copyright Registration Document

Project Name	SmartSpace OS
Project Type	Full-Stack Web Application
Repository	https://github.com/amitk1311/SmartSpace
Document Date	April 27, 2026
Version	1.0

Joint Authors

- Amit Kumar Sah
- Adarsh Ranjan
- Ashish Ranjan
- Abhishek Kumar

Technologies Used

- **Languages:** JavaScript, JSX, HTML, CSS
 - **Backend Framework:** Express.js
 - **Frontend Framework:** React
 - **Database:** MongoDB
 - **Build Tool:** Vite
 - **CSS Framework:** Tailwind CSS
-

TABLE OF CONTENTS

1. Backend Source Code
 2. Frontend Configuration
 3. Frontend Components & Pages
 4. Services & Utilities
 5. Configuration Files
-

PART 1: BACKEND SOURCE CODE

File: backend/server.js

```
import express from "express";
import cors from "cors";
import mongoose from "mongoose";
import dotenv from "dotenv";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";

dotenv.config();

const app = express();
app.use(cors({ origin: "http://localhost:5173" }));
app.use(express.json());

const JWT_SECRET = process.env.JWT_SECRET || "your-secret-key-change-in-env";

await mongoose.connect(process.env.MONGO_URI);

// ===== SCHEMAS =====

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  joinedCommunities: [{ type: mongoose.Schema.Types.ObjectId, ref: "Community" }],
  createdAt: { type: Date, default: Date.now },
});

const roomSchema = new mongoose.Schema({
  spaceId: { type: mongoose.Schema.Types.ObjectId, ref: "Space", required: true },
  name: { type: String, required: true },
  capacity: { type: Number, required: true },
  amenities: [String],
  availabilityStatus: { type: String, enum: ["available", "occupied", "unavailable"], default: "available" },
  createdAt: { type: Date, default: Date.now },
});

const spaceSchema = new mongoose.Schema({
  communityId: { type: mongoose.Schema.Types.ObjectId, ref: "Community", required: true },
  name: { type: String, required: true },
  description: { type: String, default: "" },
```

```
    type: { type: String, enum: ["Cafeteria", "Lecture Hall", "Seminar Hall", "Open Ground",
"Parking Area"], required: true },
    capacity: { type: Number, required: true },
    open: String,
    close: String,
    rooms: [{ type: mongoose.Schema.Types.ObjectId, ref: "Room" }],
    createdBy: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    createdAt: { type: Date, default: Date.now },
  });
```

```
const communitySchema = new mongoose.Schema({
  name: { type: String, required: true },
  code: { type: String, required: true, unique: true },
  description: { type: String, default: "" },
  category: { type: String, default: "General" },
  ownerId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  members: [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }],
  createdAt: { type: Date, default: Date.now },
});
```

```
const bookingSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  communityId: { type: mongoose.Schema.Types.ObjectId, ref: "Community", required: true },
  spaceId: { type: mongoose.Schema.Types.ObjectId, ref: "Space", required: true },
  roomId: { type: mongoose.Schema.Types.ObjectId, ref: "Room", default: null },
  tableNumber: { type: Number, default: null },
  bookingDate: { type: Date, required: true },
  startTime: { type: String, required: true }, // "HH : MM AM/PM"
  endTime: { type: String, required: true },   // "HH : MM AM/PM"
  status: { type: String, enum: ["confirmed", "cancelled"], default: "confirmed" },
  createdAt: { type: Date, default: Date.now },
});
```

```
const notificationSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  message: { type: String, required: true },
  isRead: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now },
});
```

```
const User = mongoose.model("User", userSchema);
const Community = mongoose.model("Community", communitySchema);
const Space = mongoose.model("Space", spaceSchema);
const Room = mongoose.model("Room", roomSchema);
const Booking = mongoose.model("Booking", bookingSchema);
const Notification = mongoose.model("Notification", notificationSchema);
```

```
// ===== MIDDLEWARE =====

const verifyToken = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ message: "No token provided" });

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.userId = decoded.userId;
    next();
  } catch (err) {
    res.status(401).json({ message: "Invalid token" });
  }
};
```

File: backend/checkData.js

```
import mongoose from "mongoose";
import dotenv from "dotenv";

dotenv.config();

await mongoose.connect(process.env.MONGO_URI);

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  createdAt: { type: Date, default: Date.now },
});

const User = mongoose.model("User", userSchema);

const users = await User.find();
console.log("===== USERS IN DATABASE =====");
console.log(JSON.stringify(users, null, 2));
console.log("\nTotal users:", users.length);

await mongoose.disconnect();
```

File: backend/package.json

```
{
```

```
"type": "module",
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
},
"dependencies": {
  "bcryptjs": "^2.4.3",
  "cors": "^2.8.6",
  "dotenv": "^17.3.1",
  "express": "^5.2.1",
  "jsonwebtoken": "^9.0.0",
  "mongoose": "^9.2.1"
},
"devDependencies": {
  "nodemon": "^3.1.11"
}
}
```

PART 2: FRONTEND - MAIN APPLICATION FILES

File: frontend/space-os/src/App.jsx

```
import { Routes, Route } from "react-router-dom";
import Landing from "../pages/Landing";
import Login from "../pages/Login";
import Signup from "../pages/Signup";
import Dashboard from "../pages/Dashboard";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Landing />} />
      <Route path="/login" element={<Login />} />
      <Route path="/signup" element={<Signup />} />
      <Route path="/dashboard" element={<Dashboard />} />
    </Routes>
  );
}
```

File: frontend/space-os/src/main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

PART 3: FRONTEND COMPONENTS

File: frontend/space-os/src/components/Navbar.jsx

```
import { useState } from "react";
import { Link } from "react-router-dom";

export default function Navbar() {
  const [open, setOpen] = useState(false);

  const closeMenu = () => setOpen(false);

  return (
    <nav className="fixed top-0 left-0 w-full h-16 bg-white shadow-md z-50">
      <div className="max-w-7xl mx-auto px-6 h-full flex items-center justify-between">

        {/* Logo */}
        <a
          href="#home"
          onClick={closeMenu}
          className="text-2xl font-bold text-emerald-600"
        >
          SmartSpace
        </a>

        {/* Desktop */}
        <div className="hidden md:flex items-center gap-8 font-medium text-gray-700">

          <a href="#features" className="hover:text-emerald-600 transition">Features</a>
        </div>
      </div>
    </nav>
  )
}
```

```
<a href="#how" className="hover:text-emerald-600 transition">How It Works</a>
<a href="#pricing" className="hover:text-emerald-600 transition">Pricing</a>
<a href="#faq" className="hover:text-emerald-600 transition">FAQ</a>
```

```
<Link to="/login" className="hover:text-emerald-600 transition">
  Login
</Link>
```

```
<Link
  to="/signup"
  className="bg-emerald-600 text-white px-5 py-2 rounded-lg hover:bg-emerald-700
transition"
>
  Get Started
</Link>
</div>
```

```
{/* Mobile Button */}
<button
  onClick={() => setOpen(!open)}
  className="md:hidden text-2xl"
>
  {open ? "X" : "≡"}
</button>
</div>
```

```
{/* Mobile Menu */}
{open && (
  <div className="md:hidden bg-white border-t shadow-lg">
    <div className="flex flex-col px-6 py-5 gap-4 font-medium text-gray-700">

      <a href="#features" onClick={closeMenu}>Features</a>
      <a href="#how" onClick={closeMenu}>How It Works</a>
      <a href="#pricing" onClick={closeMenu}>Pricing</a>
      <a href="#faq" onClick={closeMenu}>FAQ</a>

      <Link to="/login" onClick={closeMenu} className="pt-2 border-t">
        Login
      </Link>

      <Link
        to="/signup"
        onClick={closeMenu}
        className="bg-emerald-600 text-white text-center py-3 rounded-lg mt-2"
      >
        Get Started
      </Link>
```

```
        </div>
      </div>
    )}
  </nav>
);
}
```

File: frontend/space-os/src/components/Button.jsx

```
export default function Button({ children, className = "", ...props }) {
  return (
    <button
      className={`px-6 py-3 rounded-lg font-semibold transition ${className}`}
      {...props}
    >
      {children}
    </button>
  );
}
```

PART 4: FRONTEND PAGES

File: frontend/space-os/src/pages/Login.jsx

```
import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import { api, setAuthToken } from "../services/api";

export default function Login() {
  const navigate = useNavigate();
  const [form, setForm] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
    if (errors[e.target.name]) setErrors({ ...errors, [e.target.name]: "" });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

```



```

const newErrors = {};
if (!form.email.trim()) newErrors.email = "Email is required";
if (!form.password) newErrors.password = "Password is required";
if (Object.keys(newErrors).length > 0) {
  setErrors(newErrors);
  return;
}

setLoading(true);
try {
  const data = await api.auth.login(form.email, form.password);
  setAuthToken(data.token, data.user.id);
  navigate("/dashboard");
} catch (err) {
  setErrors({ server: err.message || "Login failed" });
} finally {
  setLoading(false);
}
};

return (
  <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-emerald-
50 to-teal-100">
    <div className="bg-white shadow-xl rounded-2xl p-10 w-full max-w-md">
      <h2 className="text-3xl font-bold text-center text-gray-800 mb-6">Login to
SmartSpace</h2>
      {errors.server && <div className="mb-4 bg-red-100 text-red-700 text-sm p-3 rounded-lg">
{errors.server}</div>}
      <form onSubmit={handleSubmit} className="space-y-4">
        <input
          type="email"
          name="email"
          placeholder="Gmail Address"
          value={form.email}
          onChange={handleChange}
          className={`w-full border rounded-lg px-4 py-3 focus:outline-none focus:ring-2
focus:ring-emerald-500 ${
            errors.email ? "border-red-500" : ""
          }}`
        />
        {errors.email && <p className="text-red-500 text-xs">{errors.email}</p>}
        <input
          type="password"
          name="password"
          placeholder="Password"
          value={form.password}
          onChange={handleChange}

```

```

        className={`w-full border rounded-lg px-4 py-3 focus:outline-none focus:ring-2
focus:ring-emerald-500 ${
            errors.password ? "border-red-500" : ""
        }}`
    />
    {errors.password && <p className="text-red-500 text-xs">{errors.password}</p>}
    <button
        type="submit"
        disabled={loading}
        className="w-full bg-emerald-600 text-white py-3 rounded-lg font-semibold hover:bg-
emerald-700 transition"
    >
        {loading ? "Logging in..." : "Login"}
    </button>
</form>
<p className="text-center text-gray-500 mt-6">
    Don't have account?{" "}
    <Link to="/signup" className="text-emerald-600 font-semibold">
        Sign up
    </Link>
</p>
</div>
</div>
);
}

```

File: frontend/space-os/src/pages/Signup.jsx

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { api, setAuthToken } from "../services/api";

export default function Signup() {
    const navigate = useNavigate();

    const [form, setForm] = useState({
        name: "",
        email: "",
        password: "",
        confirmPassword: ""
    });

    const [errors, setErrors] = useState({});
    const [loading, setLoading] = useState(false);
    const [success, setSuccess] = useState("");

```

```

// Validation functions
const isValidEmail = (email) => {
  const gmailRegex = /^[a-zA-Z0-9._%~]+@gmail\.com$/;
  return gmailRegex.test(email.toLowerCase());
};

const isStrongPassword = (password) => {
  return password.length >= 8;
};

const validateForm = () => {
  const newErrors = {};

  if (!form.name.trim()) newErrors.name = "Name is required";
  if (!form.email.trim()) newErrors.email = "Email is required";
  else if (!isValidEmail(form.email)) newErrors.email = "Please enter a valid Gmail address";

  if (!form.password) newErrors.password = "Password is required";
  else if (!isStrongPassword(form.password)) newErrors.password = "Password must be at least 8
characters";

  if (!form.confirmPassword) newErrors.confirmPassword = "Confirm password is required";
  else if (form.password !== form.confirmPassword) newErrors.confirmPassword = "Passwords do
not match";

  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};

const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
  if (errors[e.target.name]) {
    setErrors({ ...errors, [e.target.name]: "" });
  }
};

const handleSubmit = async (e) => {
  e.preventDefault();

  if (!validateForm()) return;

  setLoading(true);
  try {
    const data = await api.auth.register(form.name, form.email, form.password,
form.confirmPassword);

```

```

    setAuthToken(data.token, data.user.id);
    setSuccess("Account created successfully! Redirecting...");
    setForm({ name: "", email: "", password: "", confirmPassword: "" });
    setErrors({});

    setTimeout(() => {
      navigate("/dashboard");
    }, 1500);
  } catch (err) {
    setErrors({ server: err.message || "Registration failed. Please try again." });
  } finally {
    setLoading(false);
  }
};

return (
  <div className="relative min-h-screen flex items-center justify-center px-4 bg-gradient-to-br
from-white via-green-50 to-emerald-100">
    <form
      onSubmit={handleSubmit}
      className="relative w-full max-w-md bg-white/80 backdrop-blur-lg border border-white/40
shadow-xl rounded-2xl p-8 space-y-5"
    >
      <div className="text-center">
        <h2 className="text-3xl font-bold text-gray-800">
          Create Account
        </h2>
        <p className="text-gray-500 text-sm mt-1">
          Join SmartSpace platform
        </p>
      </div>
    </form>
  </div>
);
}

```

File: frontend/space-os/src/pages/Dashboard.jsx

```

import DashboardLayout from "../../layouts/DashboardLayout";

export default function Dashboard() {
  return <DashboardLayout />;
}

```

PART 5: SERVICES & UTILITIES

File: frontend/space-os/src/services/api.js

```
// API Configuration and helper functions
const API_BASE = "http://localhost:4000";

let authToken = localStorage.getItem("token");
let currentUserId = localStorage.getItem("userId");

export const setAuthToken = (token, userId) => {
  authToken = token;
  currentUserId = userId;
  localStorage.setItem("token", token);
  localStorage.setItem("userId", userId);
};

export const getAuthToken = () => authToken;
export const getCurrentUserId = () => currentUserId;
export const clearAuth = () => {
  authToken = null;
  currentUserId = null;
  localStorage.removeItem("token");
  localStorage.removeItem("userId");
};

const apiCall = async (method, endpoint, data = null) => {
  const options = {
    method,
    headers: {
      "Content-Type": "application/json",
    },
  };

  if (authToken) {
    options.headers["Authorization"] = `Bearer ${authToken}`;
  }

  if (data) {
    options.body = JSON.stringify(data);
  }

  const response = await fetch(`${API_BASE}${endpoint}`, options);
  const result = await response.json();
}
```

```

    if (!response.ok) {
      throw new Error(result.message || "API Error");
    }

    return result;
  };
};

// Auth API
export const api = {
  auth: {
    register: (name, email, password, confirmPassword) =>
      apiCall("POST", "/auth/register", { name, email, password, confirmPassword }),
    login: (email, password) =>
      apiCall("POST", "/auth/login", { email, password }),
    logout: () => clearAuth(),
  }
};

```

PART 6: CONFIGURATION FILES

File: frontend/space-os/package.json

```

{
  "name": "space-os",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.13.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.7",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",

```

```

    "autoprefixer": "^10.4.24",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "postcss": "^8.5.6",
    "tailwindcss": "^3.4.4",
    "vite": "^7.3.1"
  }
}

```

File: frontend/space-os/vite.config.js

```

import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
})

```

File: frontend/space-os/tailwind.config.js

```

/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*..{js,jsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}

```

PROJECT STRUCTURE

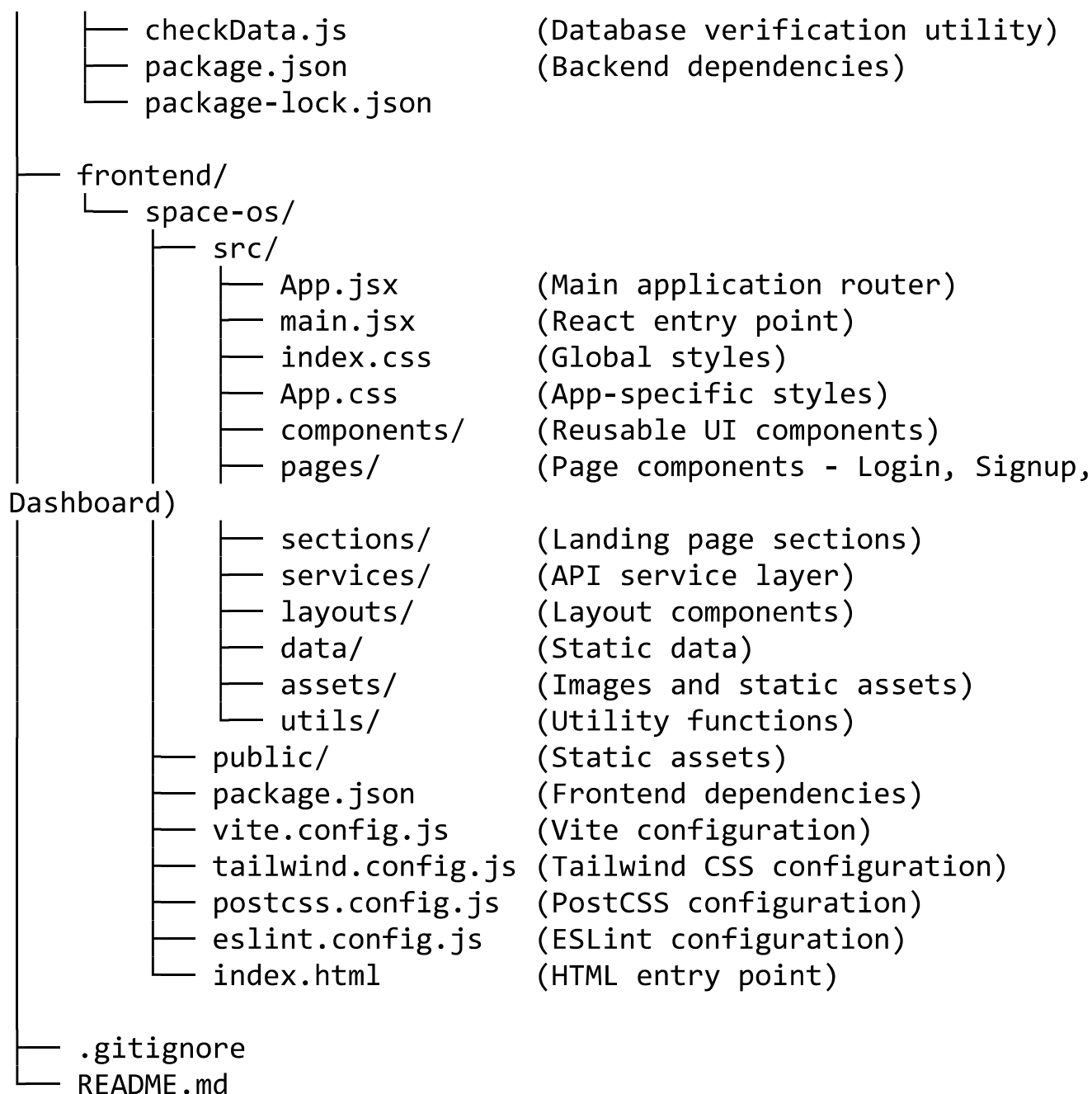
SmartSpace OS/

```

├── backend/
│   └── server.js
schemas)

```

(Main Express server with MongoDB



TECHNOLOGY STACK

Backend

- **Runtime:** Node.js
- **Framework:** Express.js 5.2.1
- **Database:** MongoDB (Mongoose 9.2.1)
- **Authentication:** JWT (jsonwebtoken 9.0.0)
- **Security:** bcryptjs 2.4.3
- **CORS:** Enabled for localhost:5173

Frontend

- **Library:** React 19.2.0

- **Build Tool:** Vite 7.3.1
 - **Router:** React Router DOM 7.13.0
 - **Styling:** Tailwind CSS 3.4.4
 - **Post-processing:** PostCSS 8.5.6
 - **Linting:** ESLint 9.39.1
-

KEY FEATURES

1. User Authentication

- Gmail-based registration and login
- JWT token-based session management
- Password hashing with bcryptjs

2. Community Management

- Create and join communities
- Community membership system
- Role-based access control

3. Space Management

- Multiple space types (Cafeteria, Lecture Hall, etc.)
- Room and amenity tracking
- Availability status management

4. Booking System

- Reserve spaces and rooms
- Time-slot based booking
- Booking history tracking

5. Responsive UI

- Mobile-first design
 - Tailwind CSS styling
 - React Router navigation
-

COPYRIGHT & LICENSING

Copyright © 2026 SmartSpace OS

All source code, documentation, and assets are the intellectual property of the copyright holder. This document is provided for copyright registration and legal protection purposes.

Repository: <https://github.com/amitk1311/SmartSpace>

ADDITIONAL PROJECT FILES

The project includes 49 total source files including:

- 6 backend/Node.js files
- 25+ React components
- 5 page components
- 8 section components
- 3+ layout components
- Configuration files (Vite, Tailwind, ESLint, PostCSS)
- Static assets and public files

This documentation represents the core intellectual property and source code of the SmartSpace OS project as of April 27, 2026.

End of Source Code Compilation